Queen Mary
University of London

EBU5376
Digital Signal Processing

Introduction to Digital Signal Processing

Module Organizer: Dr Yixuan Zou yixuan.zou@qmul.ac.uk

Outline

1. Welcome to EBU5376 Digital Signal Processing!
   - ✓ Teaching Team
   - ✓ Learning Aims
   - ✓ Syllabus
   - ✓ Assessment Structure
2. Introduction to DSP

Welcome to EBU5376 Digital Signal Processing!

- Teaching Team
  - ✓ Dr Yixuan Zou (Module Organizer)
    - yixuan.zou@qmul.ac.uk;
    - Teaches the *Telecom stream* (second half of the module)
  - ✓ Dr Richard Clegg
    - r.clegg@qmul.ac.uk
    - Teaches the *Telecom stream* (first half of the module)
  - ✓ Mr Andy Watson
    - andy.watson@qmul.ac.uk
    - Teaches the *EIE stream* (full module)

Welcome to EBU5376 Digital Signal Processing!

- Aims
  - ✓ To understand how real-world analog signals are transformed into digital signals
  - ✓ To explore real-world applications of DSP in areas like audio, communications, and imaging
  - ✓ To master essential DSP techniques, including filtering, frequency analysis, and discrete transforms, to analyze and process signals
  - ✓ To develop hands-on experience using signal processing tools to design and implement DSP solutions effectively

Learning Methods

- **Lectures**
  - ✓ 4-week block teaching
  - ✓ Delivery of
    - ✓ theoretical concepts
    - ✓ mathematical methods
    - ✓ tutorial questions
    - ✓ MATLAB examples

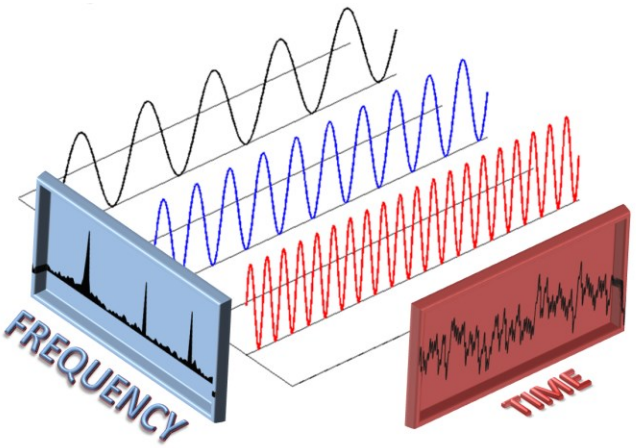
- **Assessments**
  - ✓ Class Test x 1 (closed-book)
  - ✓ Lab x 2 (MATLAB-based)
  - ✓ Exam (closed-book)

- **Textbooks**
  - ✓ O. Gazi, “Understanding Digital Signal Processing”, 2017.
  - ✓ M. Parker, “Digital Signal Processing 101: Everything you need to know to get started”, 2017.

- **QMplus**
  - ✓ Announcements
  - ✓ Student Forum
  - ✓ Lecture Slides
  - ✓ Tutorial Questions and Solutions
  - ✓ Lab materials and submissions

Syllabus

- **Week 1 – DSP Basics**
  - ✓ Introduction to Digital Signal Processing
  - ✓ Sampling and Quantization
  - ✓ Aliasing
  - ✓ Continuous Time Fourier Transform (CTFT)

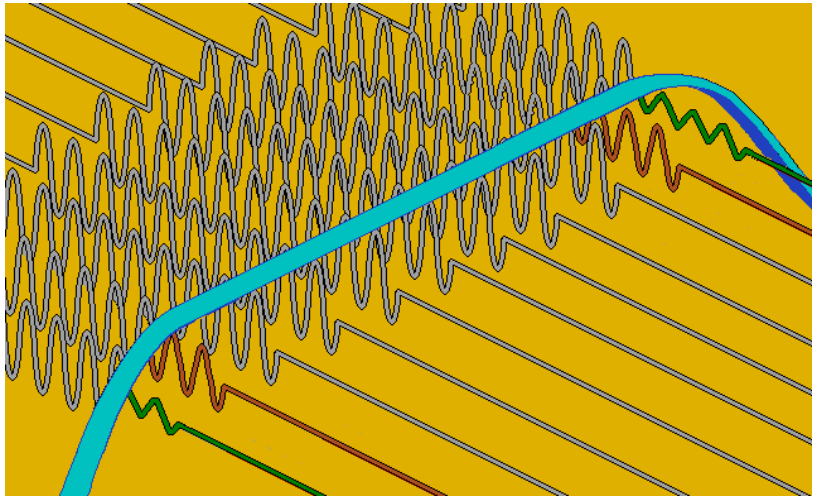


- **Week 2 – Transform Methods**
  - ✓ Discrete Time Fourier Transform (DTFT)
  - ✓ Discrete Fourier Transform (DFT)
  - ✓ Fast Fourier Transform (FFT)

Syllabus

- **Week 3 – DSP Systems**
  - ✓ Intro to DSP Systems
  - ✓ Time-domain responses
  - ✓ Frequency-domain responses
  - ✓ Stability

- **Week 4 – Filters**
  - ✓ Filters – Time-domain analysis
  - ✓ Filters – Frequency-domain analysis
  - ✓ FIR filters
  - ✓ IIR filters



Assessment Structure

➢ Coursework – 30%

2 x **Lab-based Coursework** (Total 25%)

- Written report based on *MATLAB* results
- Lab1 (10%) is based on Weeks 1&2
- Lab2 (15%) is based on Weeks 3&4

1 x **Class Test** (Total 5%)

- Written; Closed book
- 45 mins
- Based on Weeks 1&2

➢ Exam – 70%

- Written; Closed book
- 4 questions, 25 marks each
- Based on Weeks 1-4

Module Representative

- Each teaching group is required to have student Module Representatives (Reps).
- Responsibility: liaise between the lecturer and the students in that teaching group and ideally to report weekly to the lecturer about things that may have gone right or wrong
- They are also invited and encouraged to comment to the Student Staff Liaison Committee (SSLC) and to attend the SSLC meeting.
- Students who act as Module Reps gain important skills and this role can be included as an extra-curricular activity on your CV/résumé and on post-study application forms.
- Please volunteer by filling an application form via the QR code:

Outline

1. Welcome to EBU5376 Digital Signal Processing!

2. Introduction to DSP

   ✓ The History of DSP

   ✓ Modern Applications of DSP

   ✓ DSP Fundamentals

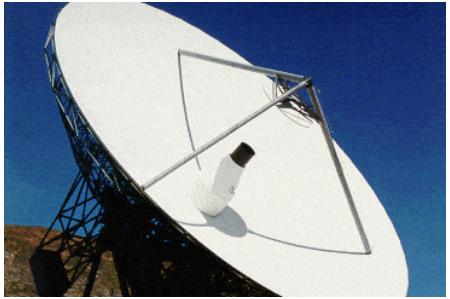
What is DSP?

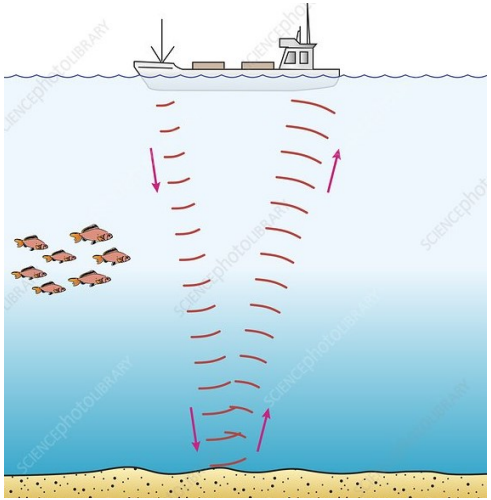
- **DSP** is the mathematics and the algorithms used to manipulate signals
- It processes signals like audio, video, temperature, or pressure after converting them into the digital form.
- For instance, noise-canceling headphones capture external noise and create an opposing waveform to cancel it—a practical application of DSP.

Why Do We Need DSP?

- **Advantages** over Analog Processing**:**
  1. **Accuracy**: DSP systems are less affected by noise, providing clearer results.
  2. **Flexibility**: Parameters can be adjusted through software rather than hardware.
  3. **Efficiency**: Facilitates efficient storage, transmission, and compression of data.
  4. **Advanced Applications**: Enables complex functionalities like real-time voice recognition and biomedical signal processing.

The Evolution of DSP





➢ 1960s

- The revolution of DSP begins in the 1960s when digital computers first became available
- Computers were expensive during this era, and DSP was limited to only a few critical applications (e.g. radar and sonar) motivated by military and government needs.

The Evolution of DSP

➢ 1980s

- The revolution of personal computer cause DSP to explode with new applications in the commercial marketplace.
- New applications include:
  - ✓ mobile telephones (e.g., echo cancellation)
  - ✓ biomedical signal processing (e.g., diagnostic imaging)
  - ✓ audio and video processing (e.g., compression and enhancement)
- DSP became a standard part of the undergraduate curriculum

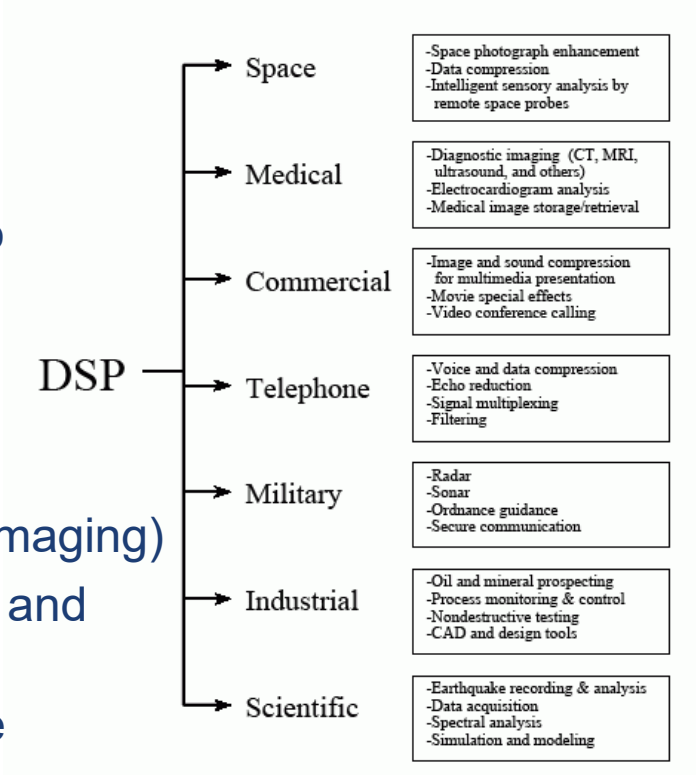


Fig.: DSP's revolution in different fields

Modern Applications of DSP – Data Compression

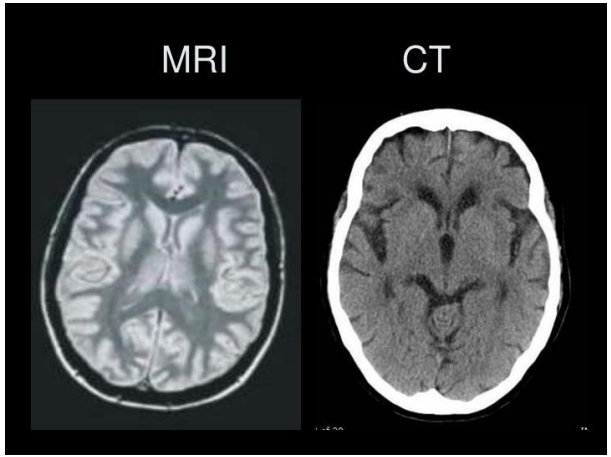
➢ JPEG Image Compression

- ✓ Standardized by Joint Photographic Experts Group (JPEG) in 1992 and remains one of the most widely used image compression formats
- ✓ It utilizes the Discrete Cosine Transform (DCT) to convert image data from the spatial domain (pixel values) to the frequency domain. Nonessential high-frequency components are then rounded or discarded to achieve compression.
- ✓ You will learn DCT in Year 3 modules (Advanced Transform Methods, Multimedia Fundamentals)

Modern Applications of DSP – Medical Imaging

- **Magnetic Resonance Imaging (MRI)**
  - ✓ The **radio frequency (RF)** transmitter emits short bursts of radio waves, which interact with hydrogen atoms in the patient's body and being picked up by the receiver.
  - ✓ The receiver digitizes the signals, applies the Fourier Transform to convert frequency-domain data into spatial-domain information, and reconstructs a detailed image of the inside of the body.

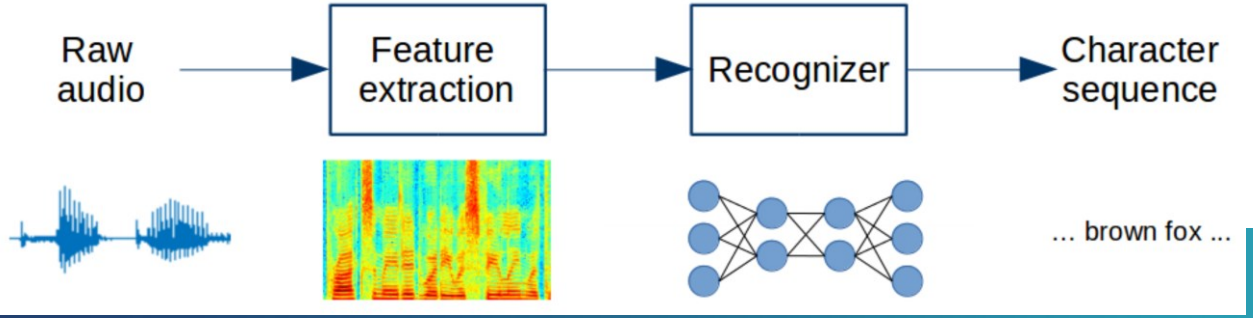
- **Computed Tomography (CT)**
  - ✓ CT emits **X-rays** and measures the attenuation as it passes through the body
  - ✓ CT does not use Fourier Transform. It relies on other DSP techniques, such as Filtered Back Projection, to construct the image



Modern Applications of DSP – Speech Recognition

➢ Speech Recognition

- ✓ A microphone captures sound waves, which are converted into analog signals and then digitized using sampling and quantization
- ✓ DSP techniques, such as Fourier transform, are used to analyze the frequency content of the speech signal and extract important features
- ✓ Extracted features are compared with a database of known patterns using machine learning algorithms to recognize spoken words.
- ✓ Example products: Siri, Alexa, Google assistant,…



Modern Applications of DSP

- **Photo/Video Manipulation**
  - ✓ DSP techniques are used in tools like Photoshop, TikTok, and other editing apps for photo and video enhancements.
  - ✓ Filtering is applied to smooth images, reduce noise, adjust color balance, or sharpen details.
  - ✓ Fourier Transform is used to process images in the frequency domain for advanced edits like blurring and edge detection.

How to carry out Digital Signal Processing?

How DSP works?

Basic Steps:

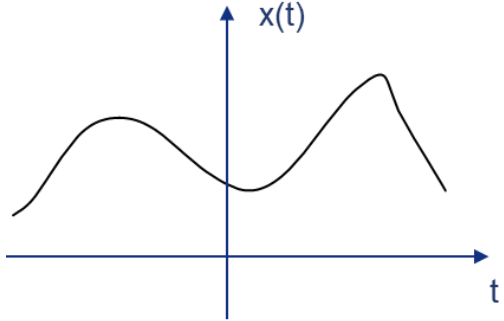
1. **Sampling**: Convert analog signals to discrete data points at regular intervals.
2. **Quantization**: Map sampled values to fixed numeric levels.
3. **Processing**: Apply filters, transforms, or algorithms to extract or modify the signal.
4. **Reconstruction**: Recreate the analog signal if required.

Basics of DSP

- What is a digital signal?
- Typical DSP system
- Mathematical Representations of Signals
- Other DSP basics

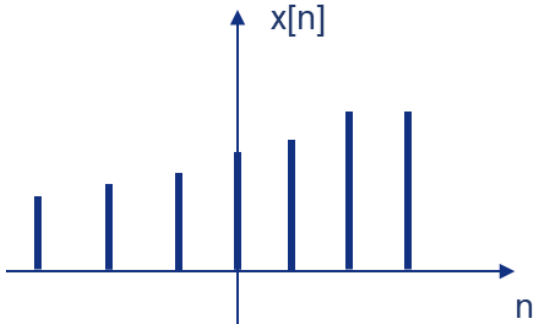
Classification of Signals

- **Continuous-time Signal, $x(t)$**



- A continuous-time signal, $x(t)$, is defined at all values of t, where t is a **continuous variable.**
- Example: $x(t) = \sin(\pi t)$
- **Real-world example**: Natural signals like the human voice

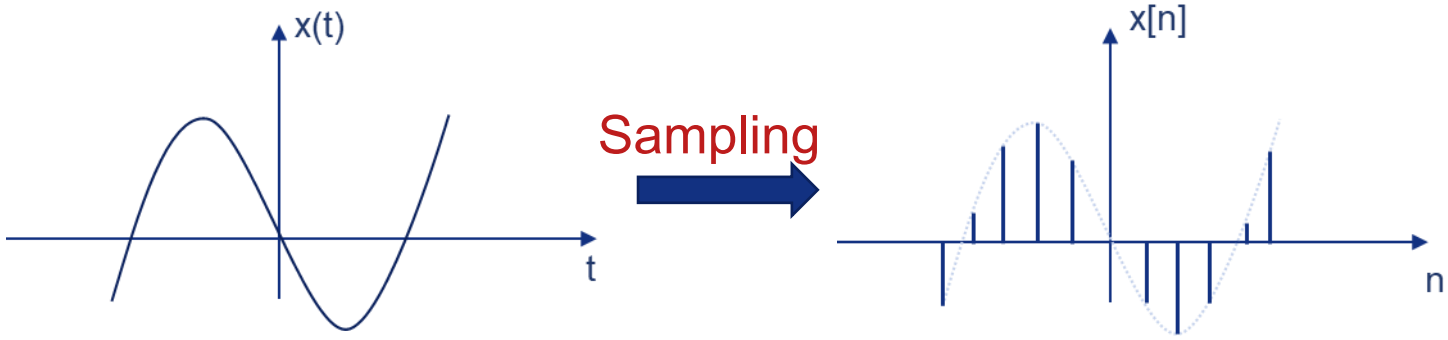
- **Discrete-time Signal, $x[n]$**



- A discrete-time signal, $x[n]$, is defined only at specific intervals, where n is a discrete variable (sample index).
- Example: $x[n] = \{1, \frac{1}{2}, \frac{1}{4}, \dots\}$
- **Real-world example**: Heart rate measured by a digital monitor at regular intervals (e.g., beats per minute).

Classification of Signals

- **Continuous-time Signal, $x(t)$**
- **Discrete-time Signal, $x[n]$**

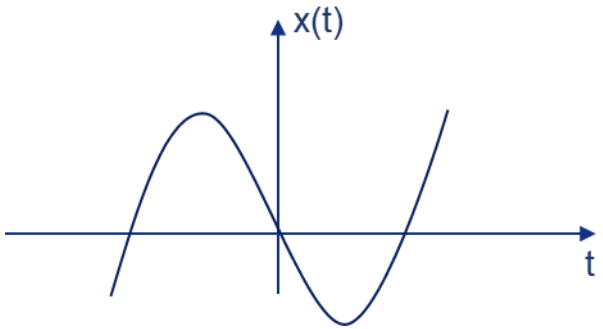


$x(t)$ $\qquad$ $x(nT_s)$

- T_s denotes the sampling interval, which is fixed during sampling
- $f_s = \frac{1}{T_s}$ denotes the sampling frequency

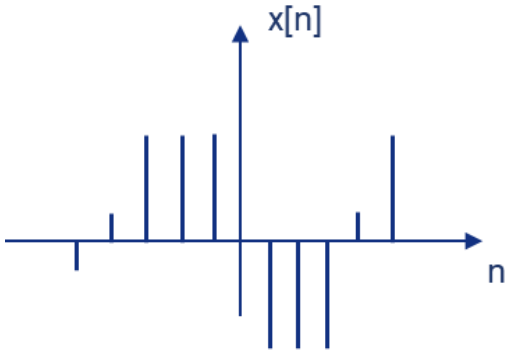
Classification of Signals

➢ Analog Signal



❖ **Analog signals** are continuous-time signals with **continuous amplitude values** that vary smoothly over time, representing **physical phenomena** such as sound, temperature, or light.

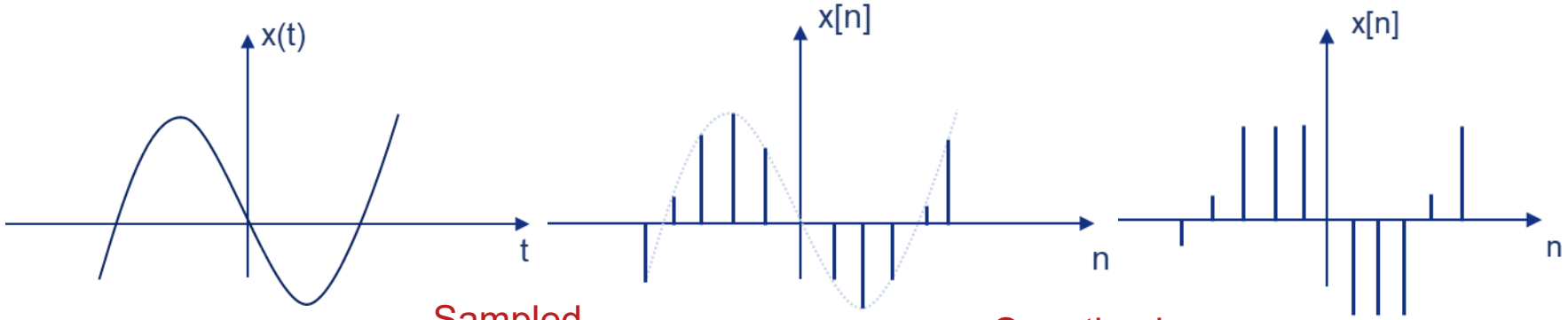
➢ Digital Signal



❖ **Digital signals** are discrete-time signals that can take on **a finite and bounded set of quantized values**, often used to represent information in binary form or other **finite-level systems**

Classification of Signals

- From Analog to Digital
  - Analog to Digital Converter (ADC)



Analog Signal $x(t)$ → Sampled → Sampled Signal $x(nT_s)$ → Quantized → Digital Signal $x[n]$

General DSP System

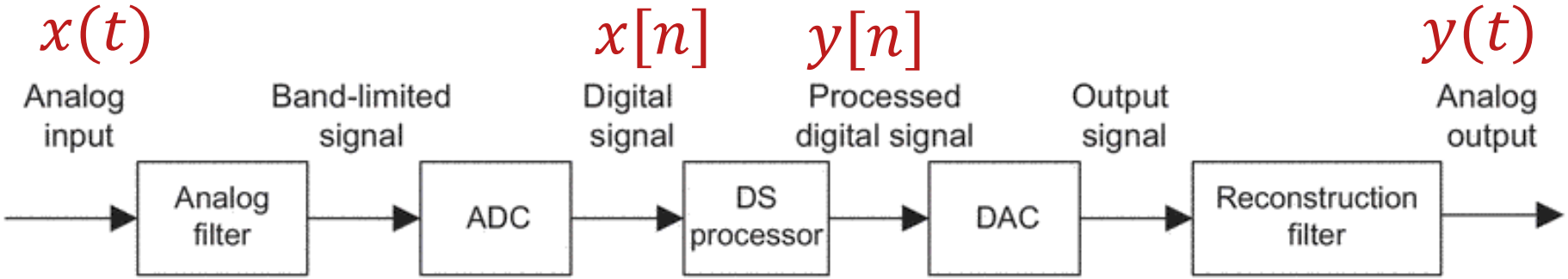
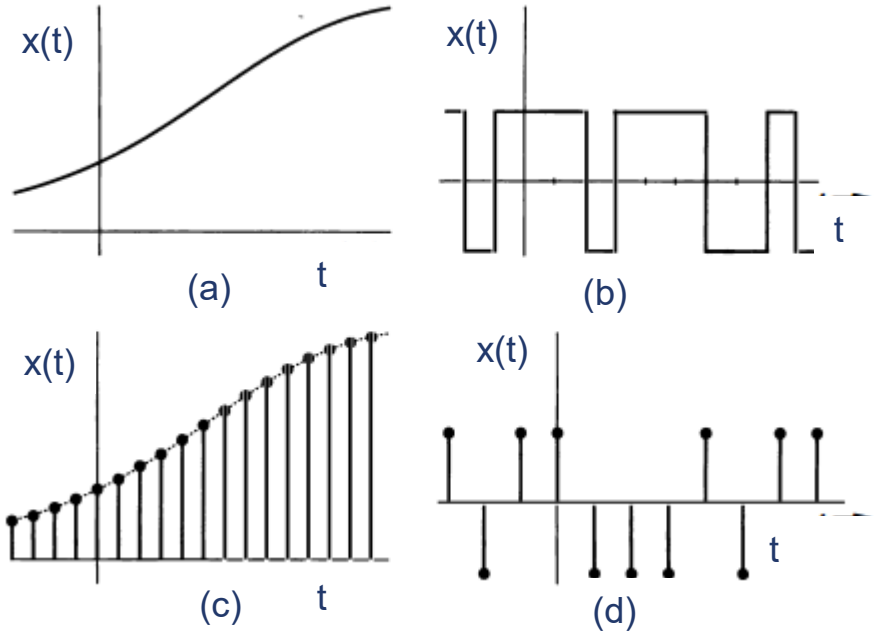


Fig. : A general DSP system.

Classification of Signals

- For each of the following signal, is this a continuous/discrete/digital/analog signal?



Basics of DSP

- What is a digital signal?
- Typical DSP system
- Mathematical Representations of Signals
- Other DSP basics

Basic Sequence Arithmetic in DSP

- **Addition/Subtraction**: Combine two sequences point by point

$$y[n] = x_1[n] \pm x_2[n]$$

  - Example: $x_1[n] = [1, 2, 3]$, $x_2[n] = [4, 5, 6] \rightarrow y[n] = [5, 7, 9]$

- **Multiplication**: Element-wise multiplication

$$y[n] = x_1[n] \cdot x_2[n]$$

  - Example: $x_1[n] = [1, 2, 3]$, $x_2[n] = [4, 5, 6] \rightarrow y[n] = [4, 10, 18]$

- **Scaling**: Multiply a sequence by a constant

$$y[n] = a \cdot x_1[n]$$

  - Example: $a = 2$, $x_1[n] = [1, 2, 3] \rightarrow y[n] = [2, 4, 6]$

Basic Sequence Arithmetic in DSP

- Right Shift (**Delay**): Delays the sequence by k units

$$y[n] = x[n-k], \qquad k \geq 0$$

  - Example: $x[n] = [1, 2, 3], k = 1 \rightarrow y[n] = [0, 1, 2, 3]$

- **Reversal**: Reverse the sequence:

$$y[n] = x[-n]$$

  - Example: $x[n] = [1, 2, 3] \rightarrow y[n] = [3, 2, 1]$

Basic Signals

- Basic signals like **impulse, step, and periodic signals (sine and cosine)** are foundational building blocks in DSP.
- **Impulse and step signals** are primarily used to study time-domain system behavior, serving as test inputs to analyze, represent, and design complex systems.
- **Periodic signals**, such as sine and cosine waves, are widely used to represent oscillatory behavior in systems like sound waves and electromagnetic waves. They also play a key role in frequency analysis, helping to break down complex signals into their frequency components.

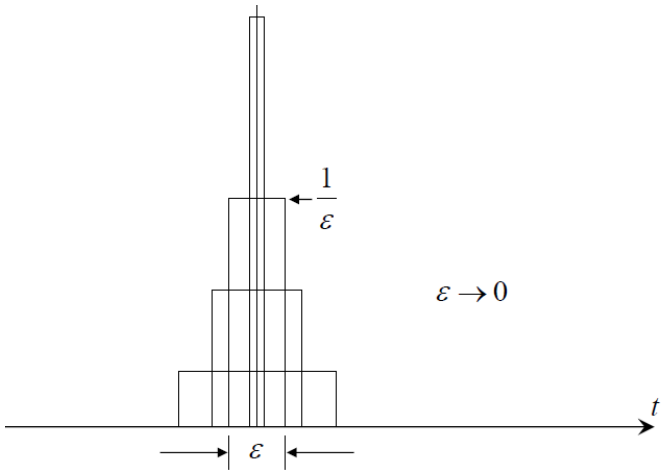
Basic Signals

Impulse Signal ($\delta(t)$ and $\delta[n]$):

- Continuous-time:

$$\delta(t) \equiv \begin{cases} 0 & t \neq 0 \\ \infty & t = 0 \end{cases} \quad \rightarrow \quad \int_{-\varepsilon}^{\varepsilon} dt\, \delta(t) \equiv 1$$

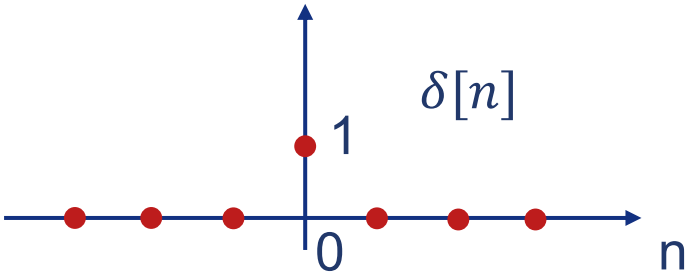
Also known as the Dirac Delta function



- Discrete-Time:

$$\delta[n] = \begin{cases} 0, & n \neq 0 \\ 1, & n = 0 \end{cases}$$

Also known as the Kronecker Delta function

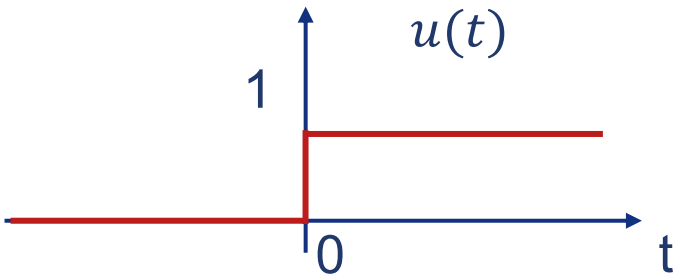


Basic Signals

Step Signal ($u(t)$ and $u[n]$):

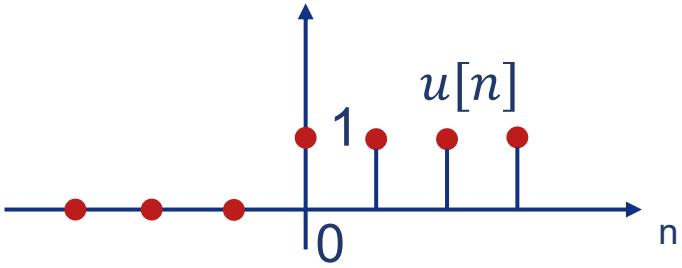
- Continuous-time:

$$u(t) = \begin{cases} 0, & t < 0 \\ 1, & t \geq 0 \end{cases}$$



- Discrete-Time:

$$u[n] = \begin{cases} 0, & n < 0 \\ 1, & n \geq 0 \end{cases}$$



Basic Signals

- Any discrete-time signal $x[n]$ can be represented as a weighted sum of time-shifted unit impulse sequences ($\delta[n]$), where the weights correspond to the signal's amplitude values at each discrete time instant.

$$x[n] = \sum_{k=-\infty}^{\infty} x[k]\delta[n-k]$$

$$\Updownarrow$$

$$x[n] = \cdots + x[-2]\delta[n+2] + x[-1]\delta[n+1] + x[0]\delta[n] + x[1]\delta[n-1] + x[2]\delta[n-2] + \cdots$$

Amplitude, Frequency, and Phase

- Given a pure **periodic** signal $x(t) = A\sin(2\pi f t + \phi)$
  - Amplitude (A):
    - The height of the wave, measured as the distance between the horizontal axis and the maximum point.
  - Frequency (f):
    - The number of cycles or oscillations per second, measured in Hertz (Hz)
  - Angular frequency ($\omega = 2\pi f$):
    - The rate of oscillation in radians per second.
  - Phase Shift (ϕ):
    - The horizontal displacement of the wave, determining where the cycle starts along the time axis

Complex Signals

- In **DSP**, signals are often represented in **complex form** to simplify analysis and operations.
- While physical signals are measured as **real quantities** (e.g., voltage or sound pressure), expressing them in a complex form allows for more efficient calculations and interpretation.
- The **imaginary part** in signal processing doesn't come from the physical measurement itself but is instead introduced as a mathematical tool to simplify analysis and operations. This can be introduced through:
  - Hilbert Transform
  - Fourier Transform
  - Modulation Techniques
  - Mathematical Models
  - Digital Filtering

Basic Signals

Complex Exponential ($e^{(\sigma+j\omega)t}$ and $e^{(\sigma+j\omega)n}$):

- Combines growth/decay and oscillation into a single expression.
- Forms the basis for understanding and analyzing signals using the **Fourier Transform**.
- Continuous-time: $x(t) = e^{(\sigma+j\omega)t} = e^{\sigma t} \cdot e^{j\omega t}$
- Discrete-Time: $x[n] = e^{(\sigma+j\omega)n} = e^{\sigma n} \cdot e^{j\omega n}$

- σ: controls the growth (σ>0) or decay (σ<0) of the signal.
- ω: determines **frequency** (rate of oscillation) **in radians**.

Basic Signals

Complex Exponential ($e^{(\sigma+j\omega)t}$ and $e^{(\sigma+j\omega)n}$)

<u>Special Cases</u>

1. Pure Oscillation ($\sigma = 0$):
   - Continuous: $x(t) = e^{j\omega t} = \cos(\omega t) + j\sin(\omega t)$
   - Discrete: $x[n] = e^{j\omega n} = \cos(\omega n) + j\sin(\omega n)$
2. Exponential Growth/Decay ($\omega = 0$):
   - Continuous: $x(t) = e^{\sigma t}$
   - Discrete: $x[n] = e^{\sigma n}$

Basics of DSP

- What is a digital signal?
- Typical DSP system
- Mathematical Representations of Signals
- Other DSP basics
  - Convolutions
  - Poles and zeros
  - Decibels

Convolution

- Convolution is a widely used mathematical operation that combines two functions or sequences to produce a third function.
- In the context of DSP, convolution describes **how an input signal interacts with another signal (often a system or filter) to produce an output signal**.
- Continuous-time Convolution:

$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau) g(t - \tau) d\tau$$

- Discrete-time Convolution:

$$(f * g)\,[n] = \sum_{m=-\infty}^{\infty} f[m] g[n - m]$$

Convolution

- The equations defined in the previous slide are known as the **linear convolution**
- In some cases, we treat signals as **periodic**.
- **Circular convolution** arises when we convolve two signals assuming they repeat infinitely in a circular manner.
- Discrete-time circular convolution:

$$(f \circledast g)[n] = \sum_{m=0}^{N-1} f[\langle n-m \rangle_N] g[m]$$

  - N is the period of $f[n]$ and $g[n]$
  - $f[\langle n-m \rangle_N]$ denotes a circular shift
  - Equivalently, $f[\langle n-m \rangle_N] = f[(n-k) \; mod \; N]$, where mod is modular operation

Poles and Zeros

- Given a function $H(z)$,
  - **Zeros**: values of z where $H(z) = 0$
  - **Poles**: values of z where $H(z)$ approaches infinity, $H(z) \to \infty$
- Mathematical representation:

$$H(z) = \frac{N(z)}{D(z)} = \frac{(z - z_1)(z - z_2) \dots}{(z - p_1)(z - p_2) \dots}$$

  - The zeros of $H(z)$ are $z_1, z_2, \dots$ and the poles of $H(z)$ are $p_1, p_2, \dots$
- Poles and zeros are used in DSP to determine how a DSP system or filter processes signals, with poles affecting the output's behavior over time (growing/shrinking/remain steady) and zeros deciding which parts of the signal are reduced or removed

Decibels, dB

- **Decibel** is a logarithmic unit used to express the ratio between two quantities, commonly in terms of power or amplitude.
- The logarithmic scale provides a compact way to represent very large or very small values.
- The **power ratio** in decibel is calculated by

$$10 \log_{10} \frac{P_1}{P_0}$$

- The **amplitude ratio** in decibel is calculated by

$$20 \log_{10} \frac{A_1}{A_0}$$

Decibels, dB

In DSP, decibels are used to express the ratio between the amplitude of the output signal and the input signal.

- The **amplitude ratio** in decibel is calculated by

$$20\log_{10}\frac{A_1}{A_0}$$

  - A_1: amplitude of the **output signal**
  - A_0: amplitude of the **input signal**

- This value is called the amplitude response in decibels (dB) of the DSP system and shows how the system modifies the signal's strength.

Decibels, dB

The **amplitude ratio** in decibel is

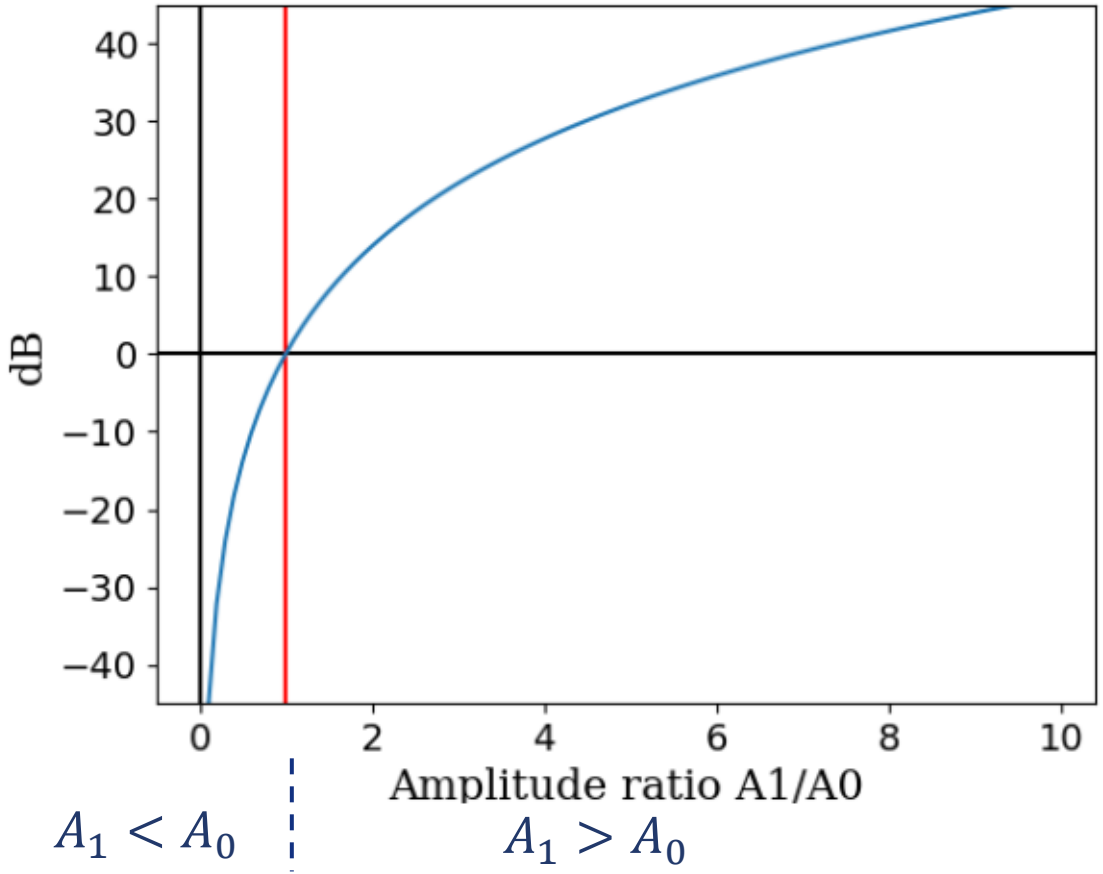
$$20\log_{10}\frac{A_1}{A_0}$$

- <u>Question</u>:
  1. What is the amplitude response (in dB) if the output and input signals have the *same amplitude*?
  2. What would the amplitude response (in dB) be like if the output amplitude is bigger/smaller than the input amplitude?

Decibels, dB

positive amplitude response in dB

negative amplitude response in dB



Queen Mary
University of London

Appendix A: MATLAB - Convolution

- The convolution in MATLAB can be done with a simple function:

```
x = [2 1 2 1];
h = [1 2 3];
clin = conv(x,h)  % linear convolution
```

```
clin = 1×6
     2     5    10     8     8     3
```

```
ccir4= cconv(x,h,4) % 4-point circular convolution
```

```
ccir4 = 1×4
    10     8    10     8
```

```
ccir6= cconv(x,h,6) % 6-point circular convolution
```

```
ccir6 = 1×6
    2.0000    5.0000   10.0000    8.0000    8.0000    3.0000
```

- The function ‘cconv()’ requires the Signal Processing Toolbox software.
- It is a very useful MATLAB toolbox that needs to be downloaded
- You will be prompt to download once you run this function

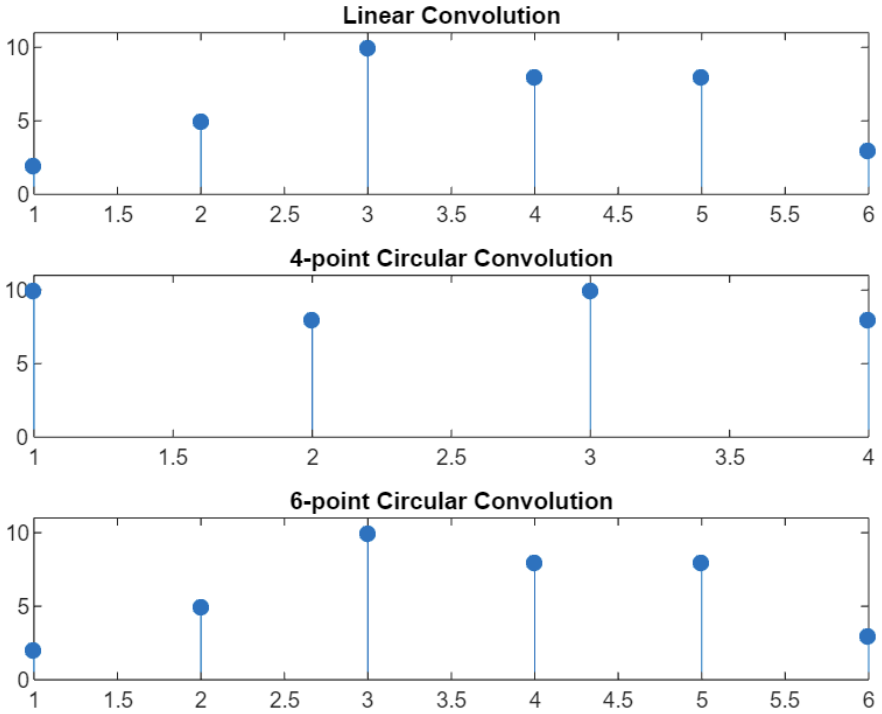
Appendix A: MATLAB - Convolution

- A good way to visualize convolution results is the stem plot:

```
subplot(3,1,1)
stem(clin,'filled')
ylim([0 11])
title('Linear Convolution')

subplot(3,1,2)
stem(ccir4,'filled')
ylim([0 11])
title('4-point Circular Convolution')

subplot(3,1,3)
stem(ccir5,'filled')
ylim([0 11])
title('6-point Circular Convolution')
```

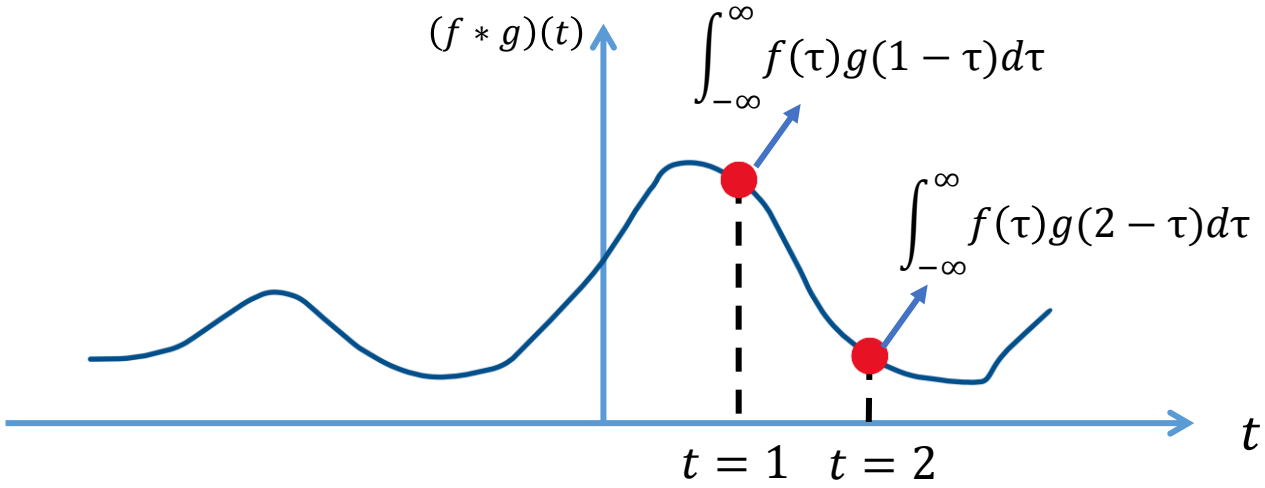


Appendix B: Convolution - Intuition

$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau)g(t-\tau)d\tau$$

- Let's understand this 'reverse sliding' movement by breaking the equation into pieces.

1. If we plot $(f * g)(t)$, each point should be an integration of the function $f(\tau)g(t-\tau)$ under a certain value of t.



We can understand the integration $\int_{-\infty}^{\infty} f(\tau)g(1-\tau)d\tau$ as the area under the curve of $f(\tau)g(1-\tau)$

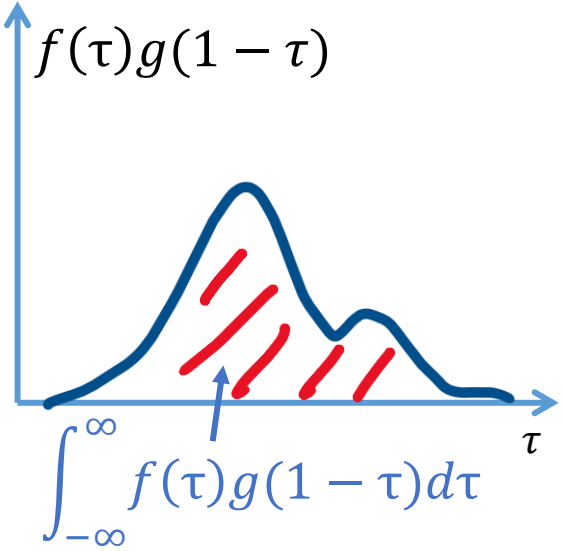
Appendix B: Convolution - Intuition

$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau)g(t-\tau)d\tau$$

2. The integration $\int_{-\infty}^{\infty} f(\tau)g(t-\tau)d\tau$ is the area under the curve of $f(\tau)g(t-\tau)$
   - Note that, in $f(\tau)g(t-\tau)$, τ is the variable and t is a constant (t=1 in the figure)

3. The function $f(\tau)g(t-\tau)$ is the product of two functions, $f(\tau)$ and $g(t-\tau)$.
   - τ is the variable in both functions

❖ *So far, we have interpreted convolution at time t as the area under the product between $f(\tau)$ and $g(t-\tau)$.*



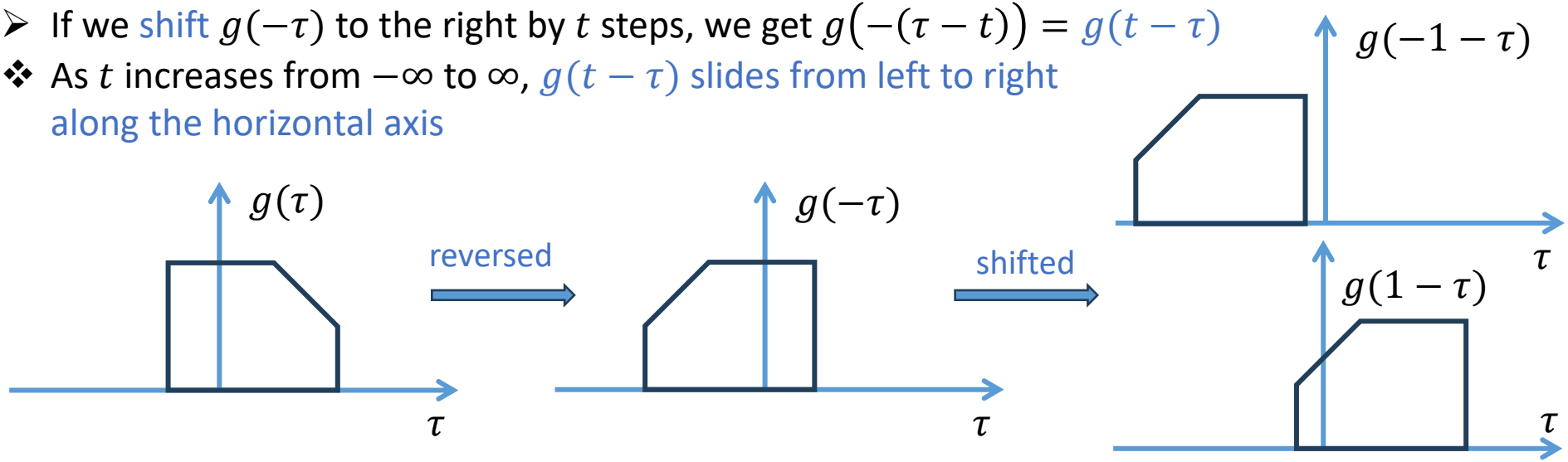
<u>Question</u>: What is the relationship between $g(\tau)$ and $g(t-\tau)$?

Appendix B: Convolution - Intuition

$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau)g(t-\tau)d\tau$$

<u>Question</u>: What is the relationship between $g(\tau)$ and $g(t-\tau)$?

- If we reverse $g(\tau)$ (flip the horizontal axis), we get $g(-\tau)$.
- If we shift $g(-\tau)$ to the right by t steps, we get $g\big(-(\tau-t)\big) = g(t-\tau)$
- As t increases from $-\infty$ to ∞, $g(t-\tau)$ slides from left to right along the horizontal axis



Appendix B: Convolution - Intuition

$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau) g(t - \tau) d\tau$$

- To conclude, the convolution between functions f and g consists of 4 actions: reverse, shift, multiply, integrate
- As t increases from $-\infty$ to ∞, the reversed function slides from left to right along the horizontal axis. When two functions overlap, we multiply the two functions and find the area under the product function as the convolution at this value of t